Введение в абстрактные типы данных

Структура данных в программировании

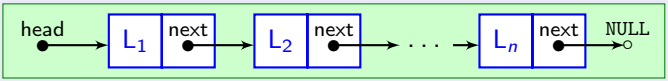
- **Структура данных (data structure):** специальный способ хранения данных удобный для построения эффективных алгоритмов
- **Абстрактные типы данных**: математические модели структур данных

Некоторые абстрактные типы данных

- Список (List)
- Стек (Stack)
- Очередь (Queue)
- Дерево (Tree)

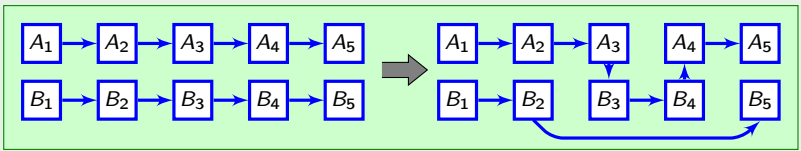
Список (List)

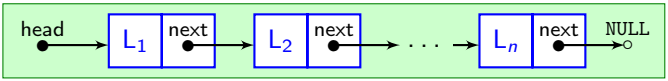
Односвязный список (Singly linked list)



- Последовательность узлов (nodes) в которых содержится:
  - ✓ L_i – сколь угодно большие, сложные данные
  - ✓ next – указатель на следующий узел, последний в списке = NULL
- Указатель на самый первый узел списка: head

☞ основное свойство: легко добавлять и удалять элементы списка





Возможная реализации списка

```
struct node_ {  // one node == list consisting of one node
   double v;
   struct node_ * next;
};
typedef struct node_ Node;
```

Функциональная спецификация: ожидаемый набор функций

1. Создание нового узла $\rightarrow$ список из одного узла
2. Вставка «другого» списка $\rightarrow$ в начало, в конец, *в заданное место*
3. Удаление одного узла $\rightarrow$ из начала, из конца, *из заданного места*
4. Подсчет числа узлов в списке: имеет сложность $O(n)$
5. *Разбиение списка на два в заданном месте*

функции односвязного списка

```
// (1) Creating a new isolated node
Node* CreateNode(double value) {
   Node* nn = (Node*) malloc(sizeof(Node)); // new node
   nn->v = value;
   nn->next = NULL;
   return nn;
}

// (2a) Inserting a list starting with node "nn"
// at the beginning of list "head"
void InsertAtHead(Node** head, Node* nn) {
   Node* tmp = nn; // search for end node of 'nn'-list
   while ( tmp->next != NULL ) { tmp = tmp->next; }
   tmp->next = *head;
   *head = nn;
}
```

```
// (2b) Inserting list "nn" at the end of list "head"
void AppendToEnd(Node** head, Node* nn) {
   if ( *head == NULL ) {
      *head = nn;
   } else {
      Node* tmp = *head; // search for end node
      while ( tmp->next != NULL ) { tmp = tmp->next; }
      tmp->next = nn; // append to the end
   }
}
// (3a) Remove one node from begining of list "head"
void DeleteFromHead(Node** head) {
   if ( *head == NULL ) {
      printf("\n%s ERROR: List is empty\n",__func__);
   } else {
      Node* tmp = *head;
      *head = tmp->next;
      free(tmp);
   }
}
```

```
// (3b) Remove one node from end of list "head"
void DeleteFromEnd(Node** head) {
   if ( *head == NULL ) {
         printf("\n%s ERROR: List is empty\n",__func__);
   } else {
      Node* tmp = *head; // search for end node
      if ( tmp->next == NULL ) { // only one node in list
         free(tmp);
         *head = NULL;
      } else {
         while ( tmp->next->next != NULL ) { tmp = tmp->next; }
         free(tmp->next);
         tmp->next = NULL;
      }
   }
}
```

```c
// (4) Print the list and return its size
size_t PrintList(Node* head) {
   size_t n = 0;
   while(head) {
      printf("%.1f -> ",head->v);
      n++;
      head = head->next;
   }
   printf("null; [size= %zu]\n",n);
   return n;
}
```

Test of singly linked list

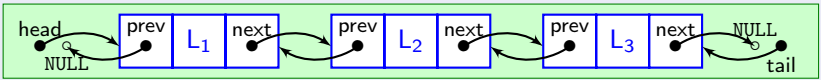
```
Node* head = NULL; // first list
for ( int i = 0; i <= 2; ++i ) {
   InsertAtHead(&head,CreateNode((double)i)); }
PrintList(head); // 2.0 -> 1.0 -> 0.0 -> null; [size= 3]

Node* list2 = NULL; // second list
for ( int i = 1; i <= 2; ++i ) {
   AppendToEnd(&list2,CreateNode(-i)); }
PrintList(list2); // -1.0 -> -2.0 -> null; [size= 2]

AppendToEnd(&head,list2);
list2 = NULL; // for safety
PrintList(head);// 2.0 -> 1.0 -> 0.0 -> -1.0 -> -2.0 -> null; [size= 5]

DeleteFromHead(&head); DeleteFromHead(&head);
PrintList(head); // 0.0 -> -1.0 -> -2.0 -> null; [size= 3]
DeleteFromEnd(&head);
PrintList(head); // 0.0 -> -1.0 -> null; [size= 2]
```

Двусвязный список (Doubly linked list)



- в каждом узле имеется ссылка на предыдущий узел: prev
- возможен указатель на последний узел списка: tail

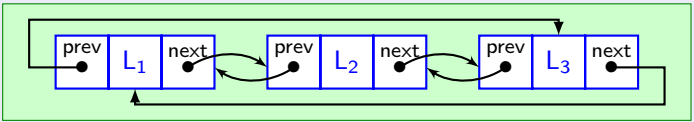
Возможная реализации двусвязного списка

```
// One node of doubly linked list
struct node_ {
   double v; // data
   struct node_ * next;
   struct node_ * prev;
};
typedef struct node_ Node;
```

```
// Doubly linked list
struct list_dl_ {
   Node* head;
   Node* tail;
};
typedef struct list_dl_ ListDL;
```

Другие разновидности списков

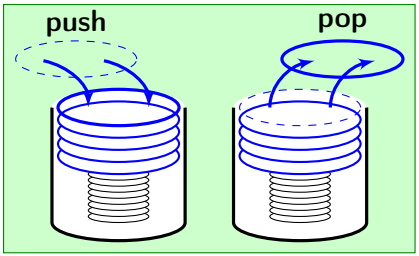
Кольцевой список (Circularly linked list)



- односвязный или двусвязный список, где next в последнем узле списка указывает на первый узел, а prev в первом узле указывает на последний узел

☞ то, что называют списком (list) в Python – массив указателей на PyObjects и не имеет отношения к рассматриваемым структурам

Стек (Stack)



Принцип организации

- Последним пришел, первым вышел (last-in, first-out: LIFO)
- Доступ, извлечение только последнего элемента

Функциональная спецификация

1. Создание стека → функция создающая пустой стек
2. Проверка является ли стек пустым
3. Добавление элемента → сохранение элемента в стеке: push
4. Извлечение верхнего элемента: pop

на основе списка

```
struct stack_sl {
   Node* head; // singly linked list
};
typedef struct stack_sl Stack;
```

```
void init(Stack* S) { S->head = nullptr; }                                    // (1)
bool is_empty(Stack* S) { return (S->head == nullptr); }                      // (2)
void push(Stack *S, double val) {                                             // (3)
   InsertAtHead(&S->head,CreateNode(val));
}
double pop(Stack* S) {                                                        // (4)
   if ( is_empty(S) ) {
      printf("\n%s ERROR: Stack is empty\n",__func__);
      exit(1);
   }
   double val = S->head->v;
   DeleteFromHead(&S->head);
   return val;
}
```

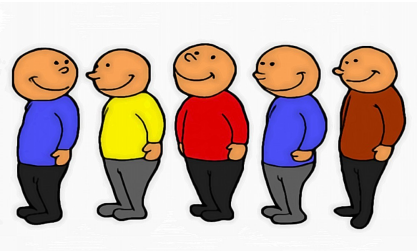
Test of Stack

```
Stack S;
init(&S);                       // initialize

for(int i = 1; i < 5; ++i) {
   push(&S, 1.1*i);             // push a few elements
}

while( !is_empty(&S) ) {
   printf("%g ", pop(&S));      // use return value in printf
}
printf("\n");                   // 4.4 3.3 2.2 1.1
```

Очередь (Queue)



Принцип организации

- Первым пришел – первым вышел (first-in, first-out: FIFO)
- Доступ, извлечение только первого элемента

Функциональная спецификация

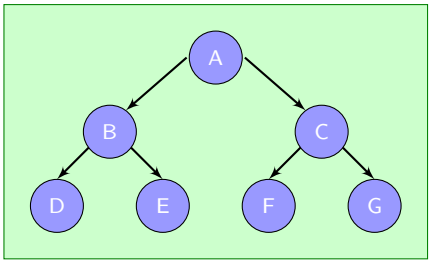
1. Создание $\rightarrow$ функция создающая пустую очередь
2. Проверка является ли очередь пустой
3. Добавление элемента в конец очереди
4. Извлечение первого элемента из очереди

Очередь на основе односвязного списка

```
struct queue_sl {
   Node* head; // singly linked list
};
typedef struct queue_sl Queue;
// Все функции как в Stack за исключением (3)
void push(Queue *Q, double val) {                                    // (3)
   AppendToEnd(&Q->head,CreateNode(val));
}
```

```
Queue Q;                        // Queue for testing
init(&Q);                       // initialize
for(int i = 1; i < 5; ++i) {
   push(&Q, 1.1*i);             // push a few elements
}
while( !is_empty(&Q) ) {
   printf("%g ", pop(&Q));      // use return value in printf
}
printf("\n");                   // 1.1 2.2 3.3 4.4
```

Дерево (Tree)



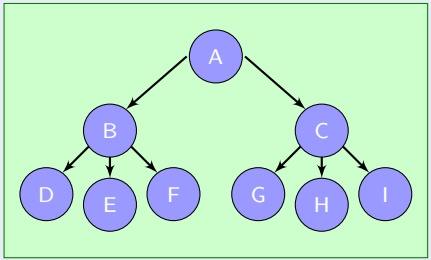
Основное свойство

☞ Дерево – иерархический набор узлов (node), связанных ребрами (edge), организованный так, что имеется единственный путь (path) от одного узла к другому

Терминология

- A – корень (`root node`)
- Соединение двух узлов – ребро (`edge`); для N узлов всего $N - 1$ ребро
- Путь – последовательность узлов и ребер между узлами: `path(AD) = A-B-D`, длина пути – полное число узлов в пути (3)
- Глубина дерева (`Depth of Tree`) – самый длинный путь из корня

Дерево общего вида



Терминология

- A,B,C – родительские (parent) узлы; B и C – родственники (siblings) и дети (childes) узла A
- Бинарное дерево – число детей любого узла не больше двух
- узлы D,E,F,G,H,I без детей – листья (leaf) или терминальные (terminal) узлы
- Каждый дочерний узел формирует поддерево (sub-tree)

1 возможная реализация «дерева распадов» на C++

☞ Структура подобна односвязному списку, но вместо `node* next` используем `vector<node*> children` — множество детей этого узла

```
struct node { // Класс для одного узла
   node(int p = 0) : pdg_code(p){}; // constructor
   int pdg_code = 0; // data - PDG particle code
   std::vector<std::unique_ptr<node>> children = {};
};
// функция для «обхода» дерева и печати элементов
void print_tree(const node& A, int lv = 0) {
   std::cout << A.pdg_code << '(';
   for ( size_t i = 0; i < size(A.children); ++i ) {
      print_tree(*A.children[i],lv+1); // recursion
   }
   std::cout << ')' << (lv > 0 ? ' ' : '\n');
}
```

Тест дерева распадов №1: $\eta \to \pi^+\pi^-\pi^0;\ \pi^0 \to \gamma\gamma$

```
auto eta = std::make_unique<node>(221); // root: eta = 221
eta->children.emplace_back(std::make_unique<node>(211)); // pi+ = 211
eta->children.emplace_back(std::make_unique<node>(-211)); // pi- =-211
eta->children.emplace_back(std::make_unique<node>(111)); // pi0 = 111

auto& pi0 = eta->children[2]; // ссылка на pi0
pi0->children.emplace_back(std::make_unique<node>(22)); // gamma = 22
pi0->children.emplace_back(std::make_unique<node>(22)); // gamma = 22

print_tree(*eta); // 221(211() -211() 111(22() 22() ) )
```

2 Другая возможная реализация «дерева распадов»

```
struct Node {
   Node(int p=0) : pdg_code(p) {}; // constructor
   int pdg_code = 0;
   std::vector<size_t> children; // position of children in Tree.nodes
};
struct Tree {
   Tree() {nodes.reserve(2);} // constructor
   std::vector<Node> nodes;
};

void print_node(const Tree& tr, size_t ch = 0) { // print Tree
   const Node& N = tr.nodes[ch];
   std::cout << N.pdg_code << '(';
   for ( auto ch : N.children ) {
      print_node(tr, ch); // recursion
   }
   std::cout << ')' << (ch > 0 ? ' ' : '\n');
}
```

Тест дерева распадов №2: $\eta \to \pi^+\pi^-\pi^0;\ \pi^0 \to \gamma\gamma$

```
Tree root;
root.nodes.push_back(221);  // [0] == root, eta = 221

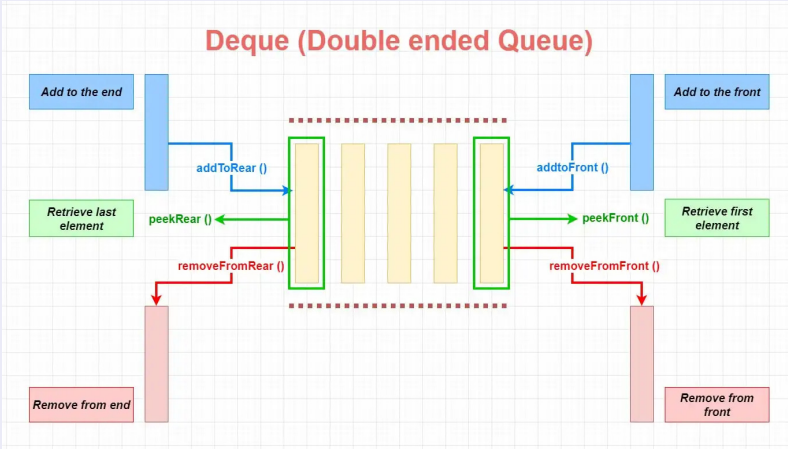
root.nodes.push_back(211);  // [1], pi+ = 211
root.nodes[0].children.push_back(1);
root.nodes.push_back(-211); // [2], pi- = -211
root.nodes[0].children.push_back(2);
root.nodes.push_back(111);  // [3], pi0 = 111
root.nodes[0].children.push_back(3);

root.nodes.push_back(22);   // [4], gamma = 22
root.nodes[3].children.push_back(4);
root.nodes.push_back(22);   // [5], gamma = 22
root.nodes[3].children.push_back(5);

print_node(root); // 221(211() -211() 111(22() 22() ) )
```

Дополнительные слайды

Дек (Double-Ended Queue): двухсторонняя очередь



☞ Обобщает концепции стека и очереди: элементы можно добавлять, удалять и читать как в начало, так и в конец DEQ

Стек на основе массива

```
struct stack_ {
   double* v;
   int max;
   int top;
};
typedef struct stack_ Stack;
```

Набор функций для стека на основе массива

```
void Init(Stack* S, unsigned int max) {                    // (1)
   S->v = (double*) malloc(max*sizeof(S->v[0]));
   S->max = max;
   S->top = 0;
}

void init(Stack* S) { Init(S,20U); }                       // (1a)

bool is_empty(Stack* S)  return S->top == 0;               // (2)
```

```
void push(Stack* S, double val) {                                    // (3)
   if( S->top >= S->max ) {
      printf("Stack full\n");
      exit(1);
   }
   S->v[ (S->top)++ ] = val;
}

double pop(Stack* S) {                                               // (4)
   if( S->top <= 0 ) {
      printf("Stack empty\n");
      exit(1);
   }
   return (S->v[--(S->top)]);
}
```